PIPE-Cypher: Automatic Enterprise Benchmark Generation for Text-to-Cypher Systems

Anonymous ACL submission

Abstract

Enterprises increasingly use property graphs and Cypher to analyze fraud, risk, identity, customer, and operational data. A useful enterprise Text2Cypher benchmark must therefore be private, refreshable, executable, and sensitive to graph-specific semantics such as relationship direction, literal grounding, and schema vocabulary. We present PIPE-Cypher, a local-model pipeline for generating natural-language-to-Cypher benchmarks inside an organization's compute boundary. PIPE-Cypher combines schema introspection, reverse-query grounding, constrained generation, deterministic Cypher governance, execution validation, diversity controls, privacy redaction, and LLM-judge review. In live public-proxy studies with local Qwen3.5-9B generation and judging, PIPE-Cypher exports 3,000 accepted FinBench/SNB examples, completes three audited ablation suites, calibrates an automated judge, onboards ICIJ Offshore Leaks, and evaluates 11 completed local downstream models. The resulting benchmark is difficult zero-shot; the strict no-signature few-shot control shows schema-specific examples can help compatible model families, while same-signature example banks are reported only as an operational upper bound.

1 Introduction

Property graphs are attractive in enterprise settings because the facts of interest are often relational paths: account transfers, identity entitlements, access chains, customer interactions, supplier dependencies, and fraud rings. Cypher gives analysts a compact language for these patterns. As LLMs become natural-language interfaces for graph analytics, an organization cannot evaluate a Text2Cypher system only on public schemas; it needs to know whether the model handles its labels, relationship directions, values, governance rules, and recurring operational questions.

For evaluation, that privacy boundary matters as much as model accuracy. A static public dataset is useful for shared comparison, but it cannot contain a bank's account taxonomy, an identity team's permission graph, or the categorical values that make a query answerable. It also cannot refresh when schemas change. Industry teams therefore need a benchmark factory: a repeatable way to turn a live property graph into a balanced, executable, privacy-aware NL-to-Cypher benchmark without sending sensitive schema or values to paid generation APIs.

PIPE-Cypher addresses this setting. The pipeline profiles a target graph, grounds question slots through reverse Cypher execution, constrains local-model generation, validates and repairs generated Cypher through deterministic gates, and uses a local LLM judge only after execution evidence is available. Its design treats Cypher correctness as a governed artifact rather than a prompt preference: relationship direction, read-only safety, exact literal use, categorical values, contextual return columns, and `RETURN DISTINCT` are checked or normalized before examples are accepted.

Contributions. We make four contributions: (1) an end-to-end local-model benchmark-factory workflow for organization-run NL-to-Cypher evaluation; (2) outcome-aware reverse grounding plus deterministic Cypher governance for read-only safety, directionality, exact literals, categorical values, contextual returns, and conservative rewrite boundaries; (3) a scaled public-proxy evaluation over FinBench, SNB, and ICIJ with ablations, judge calibration, privacy/redaction audits, and an 11-model completed local transfer stress test; and (4) reproducibility artifacts for onboarding, value sampling, benchmark refresh, evidence packaging, and appendix-level auditability.

2 Related Work

Recent Text2Cypher resources, including Mind the Query (Chauhan et al., 2025), SyntheT2C (Zhong et al., 2025), Auto-Cypher (Tiwari et al., 2025), Text2Cypher (Ozsoy et al., 2025), CypherBench (Feng et al., 2025), and the public Text2Cypher-2024 corpus (Neo4j, 2024), show that high-quality Cypher data requires schema grounding, execution checks, verification, and complexity-aware evaluation. PIPE-Cypher does not treat template filling or execution filtering as new in isolation. Its contribution is to make those pieces work as an enterprise benchmark factory: reverse grounding is outcome-aware, Cypher governance is deterministic and read-only, judge calibration is local, privacy policies are explicit, and every exported example carries an audit trail.

Mechanism	Prior Text2Cypher use	PIPE-Cypher delta
Template/ LLM synthesis	SyntheT2C and Auto-Cypher generate Question-Cypher pairs.	Outcome-aware reverse grounding binds slots through live Cypher before NL realization.
Execution validation	Auto-Cypher and Mind the Query retain executable queries.	Execution is one gate in a ledger with direction, read-only, value, non-empty, and judge evidence.
Schema/value checks	Mind the Query validates schema, runtime, and values.	Checks are packaged for private refresh with configurable value sampling and redacted exports.
Logical review	Mind the Query uses human logical validation.	Human labels calibrate a local LLM judge; they are not a generation gate.
Benchmark artifact	Public static datasets and tuning corpora.	Organization-run benchmark factory with manifests, refresh, and provenance audits.

Table 1: Prior mechanism comparison. PIPE-Cypher's contribution is the enterprise benchmark-factory composition: outcome-aware grounding, deterministic Cypher governance, local judging, privacy policy, refresh, and audit ledgers.

Mind the Query is especially relevant because it reports an Industry Track Text2Cypher dataset with schema, runtime, value, and human logical validation. PIPE-Cypher keeps that validation discipline, but changes the object of study. The artifact is not one static dataset or tuning corpus; it is the process that lets an organization generate, refresh, audit, and redact a benchmark for its own graph, local model endpoint, and value policy.

Text-to-SQL benchmarks such as Spider 2.0 (Lei et al., 2024) and BIRD (Li et al., 2023) have pushed text-to-query evaluation toward realistic database tasks and execution-based scoring, while execution-guided decoding shows why query execution is useful semantic feedback rather than a cosmetic check (Wang et al., 2018). Recent residual-skill Text-to-SQL work further shows that optimizing complementary agent skills on residual failures can improve selected accuracy across SQL dialects (Zhu et al., 2026). Graph query generation adds different failure modes: relationship direction can invert the meaning of a query, node and relationship properties live in different namespaces, and path-shaped questions can be syntactically valid while semantically ungrounded. CIKM AutoQuery (Zheng et al., 2024) motivates treating workload generation as a separate object of study from downstream model quality. LDBC FinBench (Qi et al., 2023) and SNB (P üroja et al., 2023) provide public graph workloads with financial and social-network structure, while ICIJ Offshore Leaks gives an additional public finance/compliance onboarding check.

For benchmark quality, lexical diversity alone is not enough. PIPE-Cypher combines text-generation diagnostics such as Distinct-n (Li et al., 2016) and self-BLEU-style redundancy checks (Zhu et al., 2018) with text-to-query structure metrics: schema coverage, relationship/property coverage, query-signature diversity, structural feature rates, and normalized entropy over graph/category/difficulty cells. For subset construction, PIPE-Cypher uses an MMR-style novelty objective (Carbonell and Goldstein, 1998) over Cypher signatures, template families, structural substructures, schema atoms, values, and question tokens. These metrics make a practical distinction that matters to enterprise users: a benchmark can be syntactically diverse while still overusing the same values or query templates.

3 Method

PIPE-Cypher has six stages: schema profiling, workload planning, reverse Cypher grounding, constrained Cypher generation and repair, deterministic validation and execution, and LLM-judge review. The central design choice is to make answerability and governance executable. Prompts can ask a model to respect relationship directions or exact literals, but accepted examples must prove those properties through schema checks, parser-style structure extraction, live execution, and judge review.

Schema profiling records labels, relationship types, properties, observed directions, and bounded low-cardinality categorical values. Workload planning targets eight operational categories: simple retrieval, complex retrieval, simple aggregation, complex aggregation, boolean existence, negation/difference, path/temporal transaction, and ranking/top-k. Reverse grounding runs read-only Cypher to bind slots to values that actually produce rows, reducing the common synthetic-data failure where a plausible question has no answer in the graph.

The deterministic layer enforces read-only safety, syntax shape, schema-visible labels and properties, explicit relationship types, observed relationship directions, schema-provided categorical values, and non-empty execution where required. Live execution uses read-only credentials and read-access sessions; token-level write rejection is therefore a preflight gate rather than the only safety mechanism. A lightweight Cypher analyzer extracts return aliases, variables, labels, relationship observations, risky constructs, and rewrite skip reasons so normalization is auditable rather than a silent string edit. The direction gate interprets both outgoing and incoming Cypher arrow syntax before schema checking, and rejects undirected relationship patterns so directionality is an executable constraint rather than only a prompt instruction.

4 Implementation

The implementation is a Python package with Neo4j as the experimental backend. We frame the contribution around Cypher and property graphs rather than a single vendor. Reported benchmark generation and judge review use a local Qwen3.5-9B endpoint behind a vLLM/OpenAI-compatible interface. Downstream evaluation separately tests 11 completed locally served model checkpoints spanning general instruction, code-tuned, Cypher-tuned, and Text2Cypher-tuned families. We keep the deployment pattern private: benchmark generation and evaluation run inside the organization's compute boundary without paid generation APIs.

The built-in FinBench profile is grounded in the public datagen snapshot export and includes typed node and relationship properties. Live schema inspection also records low-cardinality string values as categorical constraints for prompting and validation. The generated Neo4j import script uses uniqueness constraints for nodes and creates, rather than merges, transaction relationships so repeated account-to-account events are preserved. The SNB reference profile is grounded in the official Neo4j/Cypher headers and read-query files.

For generation, PIPE-Cypher uses seeded workload templates with deterministic reverse-binding queries and a mixed mode that prepends proven workload seeds to LLM-proposed templates. Every run records accepted and rejected candidates for yield analysis. Retrieved few-shot examples replace graph-specific values with typed placeholders, preserving query structure while reducing tenant-value leakage and memorized entity reuse. A dependency-light value grounder adds typed prompt annotations for categorical values and reverse-bound entities, covering punctuation variants, possessives, plurals, synonyms, name partials, and small typos.

Inspired by Mind the Query's prompt-setting analysis, the implementation exposes prompt profiles for schema-only, instructions-only, examples-only, examples-plus-instructions, and full governed PIPE-Cypher generation. These profiles are configured as ablation variants and must pass the same target-size and reporting-readiness standards before any result is reported.

For LLM-judge review, the implementation slices schema context to the labels, relationship types, and properties used by the candidate query. This preserves validation against the full schema while keeping local 9B prompts within context limits on larger schemas.

The deterministic Cypher layer uses production-derived constraints: schema-only prompting, exact matching, relationship direction discipline, `RETURN DISTINCT`, reserved variable rejection, categorical values, required contextual return columns, fuzzy value annotations, placeholderized retrieval examples, and parser-aware rewrite boundaries. PIPE-Cypher records parser-style structure features and skips rewrites for risky constructs such as `UNION`, `CALL`, `UNWIND`, `WHERE EXISTS`, multiple `WHERE` clauses, or reserved variables.

We also estimate seed-template capacity before

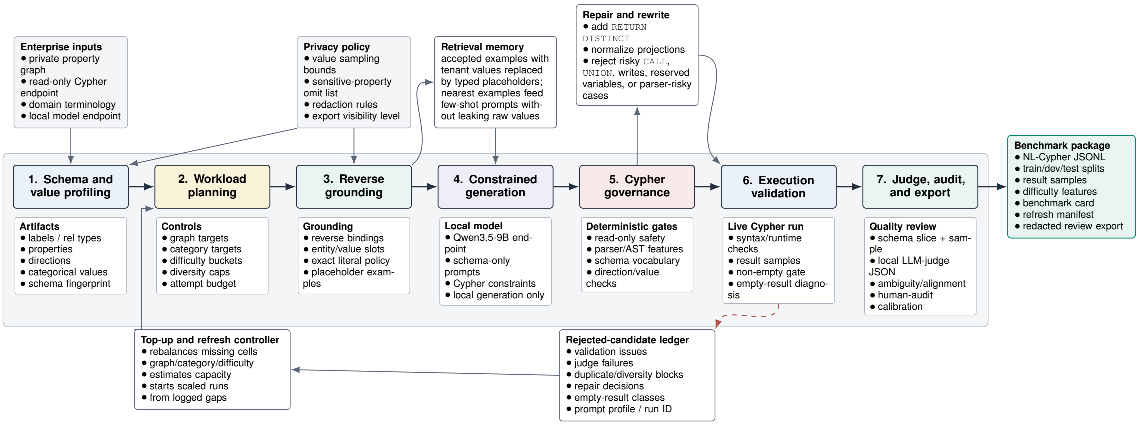

Figure 1: PIPE-Cypher benchmark generation pipeline. The key industry additions beyond static Text2Cypher dataset construction are privacy/value policies, reverse grounding, Cypher governance, execution diagnostics, local judge calibration, and benchmark export/refresh.

Gate	Evidence checked	Failure mode caught
Read-only safety	blocked Cypher write/admin tokens	destructive or operational queries
Schema validity	labels, relationship types, properties, categorical values	hallucinated graph vocabulary or values
Direction validity	observed relationship directions	reversed or invalid traversals
Cypher analysis and normalization	structure extraction, rewrite safety, `RETURN DISTINCT`	unsafe rewrites or duplicate/noisy rows
Question constraints	quoted values use exact literals	fuzzy matching of exact user values
Execution	query runs and returns rows	syntactic validity without answerability
LLM judge	ambiguity, semantic alignment, schema use	executable but semantically weak examples

Table 2: PIPE-Cypher candidate acceptance gates.

full generation. This check caught and fixed a scale blocker: reverse-binding execution was hard-capped to 10 rows, and no-slot negation/ranking seeds could not support full category targets. PIPE-Cypher now uses the configured binding limit and includes slotted negation and ranking seeds for both graphs.

For enterprise onboarding, PIPE-Cypher includes a deployment template for a company's own graph, read-only credentials, local model endpoint, schema introspection, privacy policy, dry run, scaled run, audit, and redacted export. We validate the pattern on three public proxy graphs rather than a proprietary tenant deployment. Configurable value-sampling policies bound which low-cardinality graph values enter schema prompts, and redacted exports replace quoted literals, entity values, and string-valued result samples with stable placeholders for broader internal review. The ICIJ onboarding run further motivated schema-derived sparse-category templates: PIPE-Cypher now derives relationship-count, anti-join, and top-k templates from observed labels, relationship directions, and safe low-cardinality properties, then grounds slot values with outcome-aware reverse Cypher.

Graph	Target/category	Binding limit	Min seed capacity	All categories meet target
FinBench	250	300	300	Yes
SNB	125	200	200	Yes

Table 3: Built-in seed-template capacity after removing the reverse-binding 10-row cap and adding slotted negation/ranking seeds. Capacity is a theoretical scale-readiness check; final examples still must pass validation, execution, diversity, and judge gates.

5 Experiments

Research questions. We evaluate PIPE-Cypher around four questions that matter for an industry benchmark generator: RQ1, can a local-model pipeline produce a balanced executable benchmark over live property graphs? RQ2, do Cypher-specific governance and grounding steps make generation reliable at scale? RQ3, does the resulting benchmark expose meaningful downstream Text2Cypher failures rather than merely checking syntax? RQ4, can the same machinery onboard a new public enterprise-style graph without hard-coding FinBench or SNB?

The generated benchmark contains 3,000 ac-

Graph	Candidates	Accepted	Acceptance	Categories at target
FinBench	3,405	2,000	0.587	8/8
SNB	1,520	1,000	0.658	8/8
Total	4,925	3,000	0.609	16/16

Table 4: Full live generation with local Qwen3.5-9B. Candidate counts include the initial sequential run and category-specific recovery top-ups.

cepted examples: 2,000 from LDBC FinBench and 1,000 from LDBC SNB, balanced over eight workload categories. We report three completed FinBench/SNB ablation suites: target-50, corrected target-100, and a seed-17 target-50 repeat. Downstream Text2Cypher evaluation uses live execution accuracy and answer-set F1 as primary metrics; reference-based text metrics are supported for debugging only and reported in the appendix. We additionally report ICIJ Offshore Leaks as a third public finance/compliance onboarding proxy.

6 Results

RQ1: executable benchmark generation. The full live run produced 3,000 accepted examples from 4,925 candidates using local Qwen3.5-9B for generation and judging. Category-specific recovery top-ups filled the only under-target categories from the initial sequential run. Every exported example passed read-only, syntax, schema, execution, non-empty result, and judge gates.

The accepted records are exported with stable identifiers, train/dev/test splits, result samples, gate metadata, aggregate statistics, and a manifest hash; the appendix gives the full artifact distribution.

Diversity and residual concentration. We report diversity as an audit surface rather than a single headline score. The full export has perfect category balance, near-perfect difficulty balance, 1,115 unique grounded entity values, and exact quotation of grounded values in 82.6% of examples with entity bindings. Query-signature diversity remains low because seeded graph-grounded templates do substantial work, but a diversity-governed selector improves structural coverage, adjusted Distinct-2, query-signature ratio, and property coverage at the same graph/category target (Appendix Table 22). PIPE-Cypher therefore produces a balanced executable benchmark while making residual template and value concentration visible for refresh.

RQ2: governed generation. The full-run failure taxonomy is concentrated in duplicate/diversity controls and empty execution results; only 2/4,925 candidates were schema-invalid after governance.

Split	Examples	Parse	Schema	Exec. success	Exec. acc.	Answer F1
Live full test	296	0.963	0.916	0.611	0.189	0.189

Table 5: Downstream Text2Cypher evaluation for local Qwen3.5-9B on the exported full benchmark test split.

A rewrite audit found that all reported-run candidates were already identical to their normalized Cypher, so accepted examples do not depend on semantics-changing rewrites. The ablations are therefore a reliability study of the execution-grounded core: in the target-100 suite, every non-unconstrained graph/setting cell reached all eight category targets, while unconstrained generation did not provide balanced executable coverage. Across the three evidence-ready suites, target-normalized coverage is 1.000 for every non-unconstrained cell.

Judge calibration. An 80-row post-hoc human audit sampled accepted and rejected candidates across both graphs and all categories. Agreement is 80.0%, Cohen's $\kappa = 0.60$, judge precision/specificity are 1.00, recall is 0.714, and no false accepts appear in the sample. The judge is conservative and protects accepted-example quality; human labels calibrate the gate but do not participate in generation.

RQ3: downstream stress test. We evaluate downstream Text2Cypher by giving each model schema text and the question, then scoring live execution on FinBench or SNB. This is where a benchmark earns its keep: many model outputs parse, mention plausible schema, and still answer the wrong graph question. In an 11-model completed local transfer suite, zero-shot execution accuracy ranges from 0.000 to 0.203 (mean 0.036). The primary few-shot generalization result is the scored no-signature control, which excludes exact query-signature matches and near-duplicate questions and raises mean accuracy to 0.200. Ordered and random same-category example banks reach 0.269/0.267, but are reported as operational upper bounds because they often share query signatures (Appendix Tables 16–18).

RQ4: third-graph onboarding. ICIJ onboarding reaches 800 accepted examples from 983 candidates on a 2.0M-node, 3.3M-edge public finance/compliance graph, with 100 examples in every category. This is third-public-proxy evidence, not proof of private-tenant coverage, but it exercises schema-derived relationship-count, anti-join, and top-k templates beyond the two LDBC study workloads.

7 Industry Use

PIPE-Cypher is intended to be rerun when an enterprise graph changes. The artifact records schema snapshot, graph profile, model identifier, validation gates, execution samples, judge scores, difficulty features, and source run for every example. Teams can refresh after schema changes, compare models on the same held-out split, inspect failures by category, build a schema-specific question-answer demonstration bank for retrieval or adaptation, and export redacted review packets for governance. Deployment requires read-only graph credentials, schema introspection, a bounded value policy, a local endpoint, a dry run, scaled generation, judge calibration, and redacted export. The FinBench/SNB/ICIJ study validates this pattern on public proxy graphs while making the proprietary-tenant gap explicit.

8 Conclusion

PIPE-Cypher reframes Text2Cypher benchmarking as a private, repeatable enterprise workflow. The main lesson is that benchmark generation improves when Cypher constraints become executable gates: reverse grounding makes questions answerable, deterministic governance catches unsafe or schema-invalid queries, execution exposes empty or brittle candidates, diversity diagnostics reveal concentration, and a calibrated local judge provides a conservative semantic filter. This produces a benchmark that is not only larger, but more useful: it is balanced, auditable, refreshable, and able to reveal downstream model failures that syntax-only evaluation would hide.

Limitations

Execution validity does not guarantee semantic correctness. The completed 80-row, single-audit-sheet calibration suggests a conservative judge with no observed false accepts in the labeled sample, but the confidence interval is necessarily wider than the point estimate and larger multi-annotator audits may reveal additional failure modes. FinBench, SNB, and ICIJ are public enterprise-style proxies rather than a proprietary tenant graph, so we test the onboarding pattern but not every deployment constraint of a real organization. The full export is balanced by graph, category, and difficulty, yet query-signature diagnostics show residual template concentration from the seeded, execution-grounded generation strategy. PIPE-Cypher deliberately disallows or skips risky Cypher constructs such as writes, undirected relationships, `UNION`, `CALL`, `UNWIND`, and parser-risky rewrites; this is a safe benchmark-generation subset, not complete coverage of every production Cypher idiom. The downstream few-shot result should be read as graph-specific example-bank conditioning: the no-signature control is the leakage-aware result, while ordered and random same-category demonstrations are upper-bound operating conditions that often share query signatures. Tenant-specific fine-tuning remains an engineering path enabled by the artifact, not a completed deployment claim here. The redaction audit checks exact residuals for known value-bearing strings, but it is not a full PII classifier and does not make schema names confidential.

Ethics Statement

PIPE-Cypher is designed for private benchmark generation under local-model deployment constraints. Organizations using it should restrict benchmark artifacts to authorized users, review sampled values for sensitive content, and document whether judge calibration relied on human annotators.

References

Satanjeev Banerjee and Alon Lavie. 2005. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, pages 65–72, Ann Arbor, Michigan. Association for Computational Linguistics.

Jaime Carbonell and Jade Goldstein. 1998. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336. ACM.

Vashu Chauhan, Shobhit Raj, Shashank Mujumdar, Avirup Saha, and Anannay Jain. 2025. Mind the query: A benchmark dataset towards Text2Cypher task. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1890–1905, Suzhou, China. Association for Computational Linguistics.

Yanlin Feng, Simone Papicchio, and Sajjadur Rahman. 2025. CypherBench: Towards precise retrieval over full-scale modern knowledge graphs in the LLM era. *Preprint*, arXiv:2412.18702.

Matthew A. Jaro. 1989. Advances in record-linkage methodology as applied to matching the 1985 census of tampa, florida. *Journal of the American Statistical Association*, 84(406):414–420.

Moussa Kamal Eddine, Guokan Shang, Antoine Tixier, and Michalis Vazirgiannis. 2022. FrugalScore: Learning cheaper, lighter and faster evaluation metrics for automatic text generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1305–1318, Dublin, Ireland. Association for Computational Linguistics.

Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. 2024. Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. *Preprint*, arXiv:2411.07763.

Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems*.

Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and Bill Dolan. 2016. A diversity-promoting objective function for neural conversation models. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 110–119, San Diego, California. Association for Computational Linguistics.

Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain. Association for Computational Linguistics.

Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press.

Neo4j. 2024. text2cypher-2024v1: Consolidated text-to-Cypher dataset. Hugging Face dataset.

Makbule Gulcin Ozsoy, Leila Messallem, Jon Besga, and Gianandrea Minneci. 2025. Text2Cypher: Bridging natural language and graph databases. In *Proceedings of the Workshop on Generative AI and Knowledge Graphs*, pages 100–108, Abu Dhabi, UAE. International Committee on Computational Linguistics.

Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA. Association for Computational Linguistics.

David P
üroja, Jack Waudby, Peter Boncz, and G
'abor Sz
'arnyas. 2023. The LDBC social network benchmark interactive workload v2: A transactional graph query benchmark with deep delete operations. *Preprint*, arXiv:2307.04820.

Shipeng Qi, Heng Lin, Zhihui Guo, G
'abor Sz
'arnyas, Bing Tong, Yan Zhou, Bin Yang, Jiansong Zhang, Zheng Wang, Youren Shen, Changyuan Wang, Parviz Peiravi, Henry Gabb, and Ben Steer. 2023. The LDBC financial benchmark. *Preprint*, arXiv:2306.15975.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas. Association for Computational Linguistics.

Aman Tiwari, Shiva Krishna Reddy Malay, Vikas Yadav, Masoud Hashemi, and Sathwik Tejaswi Madhusudhan. 2025. Auto-cypher: Improving LLMs on cypher generation via LLM-supervised generation-verification framework. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2: Short Papers*, pages 623–640, Albuquerque, New Mexico. Association for Computational Linguistics.

Chenglong Wang, Kedar Tatwawadi, Marc Brockschmidt, Po-Sen Huang, Yi Mao, Oleksandr Polozov, and Rishabh Singh. 2018. Robust text-to-SQL generation with execution-guided decoding. *Preprint*, arXiv:1807.03100.

William E. Winkler. 1990. String comparator metrics and enhanced decision rules in the Fellegi-Sunter model of record linkage. In *Proceedings of the Section on Survey Research Methods*, pages 354–359. American Statistical Association.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations*.

Xiuwen Zheng, Arun Kumar, and Amarnath Gupta. 2024. Generating cross-model analytics workloads using LLMs. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 4303–4307. ACM.

Zijie Zhong, Linqing Zhong, Zhaoze Sun, Qingyun Jin, Zengchang Qin, and Xiaofan Zhang. 2025. SyntheT2C: Generating synthetic data for fine-tuning large language models on the Text2Cypher task. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 672–692, Abu Dhabi, UAE. Association for Computational Linguistics.

Jiongli Zhu, Haoquan Guan, Parjanya Prajakta Prashant, Nikki Lijing Kuang, Seyedeh Baharan Khatami, Canwen Xu, Xiaodong Yu, Yingyu Lin, Zhewei Yao, Yuxiong He, and Babak Salimi. 2026. Residual skill optimization for text-to-sql ensembles. *Preprint*, arXiv:2605.21792.

Yaoming Zhu, Sidi Lu, Lei Zheng, Jiaxian Guo, Weinan Zhang, Jun Wang, and Yong Yu. 2018. Texygen: A benchmarking platform for text generation models. In *The 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1097–1100. ACM.

A Experimental Setup and Exported Benchmark

All extended results use the same live FinBench/SNB benchmark export as the main results unless a table explicitly names an ablation suite or the ICIJ onboarding run. The setup tables below fix the scale, graph mix, split, and validation totals for the artifact used in the downstream and diversity analyses.

A.1 Benchmark Artifact Summary

Tables 6, 7, and 8 give the run configuration, export manifest summary, and gate/distribution details for the 3,000-example benchmark.

Setting	Value
Accepted examples	3,000
Primary graph	LDBC FinBench, 2,000 examples
Secondary graph	LDBC SNB, 1,000 examples
Categories	8 balanced categories, 375 each
Generation/judge model	Local Qwen3.5-9B
Execution backend	Neo4j Community, two live databases
Judge audit packet	80 sampled accepted/rejected pairs

Table 6: Full live experimental setup used for the generated benchmark artifact.

Export artifact	Examples	FinBench	SNB	Train	Dev	Test
Live full benchmark	3,000	2,000	1,000	2,408	296	296

Table 7: Accepted live full benchmark package with stable IDs, gate metadata, result samples, statistics, and manifest hash `cf274344be2abe7e`.

Artifact property	Value
Categories	8 balanced categories
FinBench/category	250
SNB/category	125
Difficulty split	1,569 easy / 1,431 medium
Used labels / rel. types	16 / 17
Read/syntax/schema/exec/judge gates	3,000/3,000

Table 8: Distribution and gate summary for the exported full benchmark artifact.

B Governed Generation Evidence

The central reliability question is whether reverse grounding and deterministic Cypher governance can fill every planned graph/category cell at target scale.

B.1 Target-100 Stress Baseline

Figure 2 and Table 9 show the target-100 stress baseline. The unconstrained rows are deliberately harsh: plausible raw local-model generations do not produce balanced, executable benchmark coverage. The governed variants fill every target cell on both graphs.

Setting	Graph	Attempts	Records	Accepted	Acceptance	Categories at target
Unconstrained LLM	FinBench	422	422	200	0.474	2/8
Unconstrained LLM	SNB	2,000	2,000	50	0.025	0/8
Reverse-only	FinBench	815	815	800	0.982	8/8
Reverse-only	SNB	820	820	800	0.976	8/8
Validators+repair	FinBench	833	833	800	0.960	8/8
Validators+repair	SNB	824	824	800	0.971	8/8
No retrieval	FinBench	819	819	800	0.977	8/8
No retrieval	SNB	809	809	800	0.989	8/8
No rewrite	FinBench	826	826	800	0.969	8/8
No rewrite	SNB	834	834	800	0.959	8/8
No LLM judge	FinBench	812	812	800	0.985	8/8
No LLM judge	SNB	828	828	800	0.966	8/8
Full PIPE-Cypher	FinBench	824	824	800	0.971	8/8
Full PIPE-Cypher	SNB	824	824	800	0.971	8/8

Table 9: Live target-100 ablation evidence with local Qwen3.5-9B. Governed graph runs target 100 accepted examples per category; the unconstrained row is a stress baseline reported with explicit attempt accounting.

B.2 Gate-Level Quality

Yield alone is not enough. A benchmark factory must show which gates keep bad examples out. Table 10 and Figure 3 report read-only, syntax, schema, execution, and judge/post-hoc rates for the same target-100 suite. Read-only and syntax checks saturate after governance; execution and semantic judging expose the remaining differences.

Setting	Graph	Read-only	Syntax	Schema	Exec.	Judge/post-hoc
Unconstrained LLM	FinBench	1.000	1.000	0.806	0.723	0.557
Unconstrained LLM	SNB	1.000	0.998	0.599	0.478	0.144
Reverse-only	FinBench	1.000	1.000	0.982	0.982	0.982
Reverse-only	SNB	1.000	1.000	0.998	0.998	0.976
Validators+repair	FinBench	1.000	1.000	1.000	1.000	0.960
Validators+repair	SNB	1.000	1.000	0.998	0.998	0.971
No retrieval	FinBench	1.000	1.000	1.000	1.000	0.977
No retrieval	SNB	1.000	1.000	0.998	0.998	0.989
No rewrite	FinBench	1.000	1.000	1.000	1.000	0.969
No rewrite	SNB	1.000	1.000	0.998	0.998	0.959
No LLM judge	FinBench	1.000	1.000	1.000	1.000	0.985
No LLM judge	SNB	1.000	1.000	0.998	0.998	0.966
Full PIPE-Cypher	FinBench	1.000	1.000	1.000	1.000	0.971
Full PIPE-Cypher	SNB	1.000	1.000	0.998	0.998	0.971

Table 10: Quality-gate rates for the live target-100 ablation suite. Rates are computed over all generated records in each graph/setting; for no-judge settings, the judge column is a post-hoc scoring diagnostic.

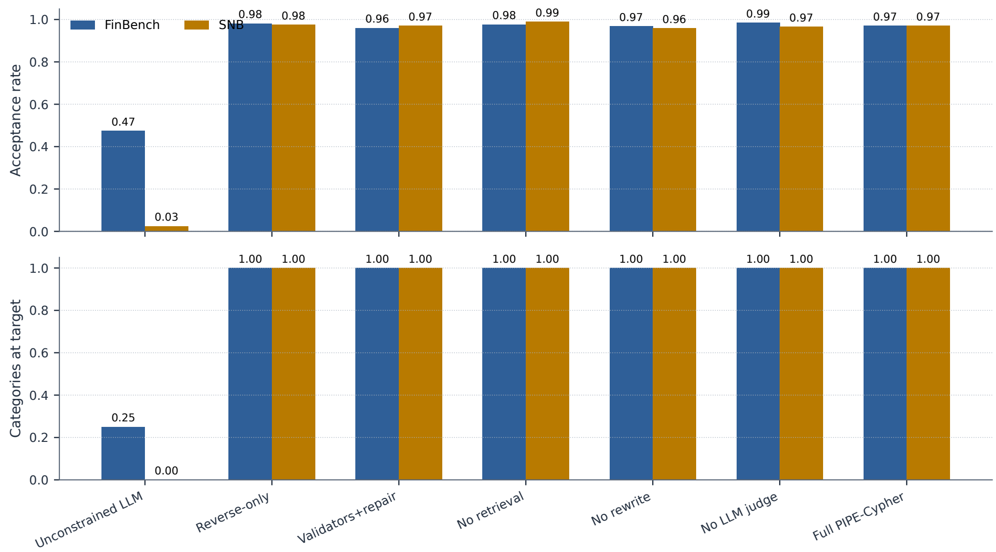

Figure 2: Target-100 ablation yield on FinBench and SNB. Unconstrained local generation is reported as a stress baseline with explicit attempt accounting; the execution-grounded governed variants reach all eight workload-category targets on both graphs. The result supports reliability of the grounded governance core without implying that every optional module is required for yield.

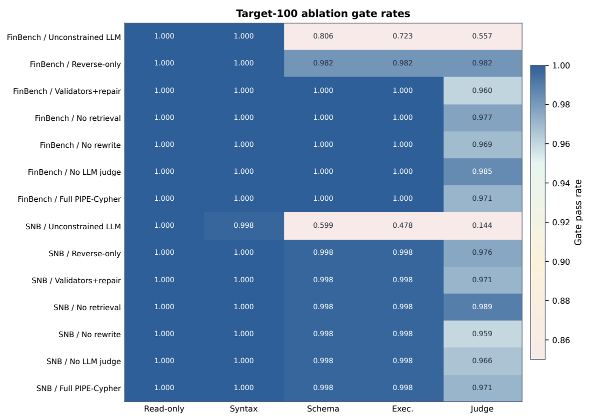

Figure 3: Gate-rate heatmap for the audited target-100 ablation suite. Deterministic read-only and syntax gates are saturated; execution and judge rates expose the remaining quality differences across graph and pipeline variants.

B.3 Repeated-Suite Stability

The same pattern holds across target sizes and seeds. The repeated-suite comparison normalizes by each suite's planned target, so it measures stability rather than rewarding larger raw counts. Full PIPE-Cypher and the governed ablations reach complete target coverage on both graphs, which is what a repeatable benchmark factory should do.

Setting	Graph	Suites	Target cov.	Acceptance	Cat. target	Exec.	Judge
No LLM judge	FinBench	3/3	1.000	0.994 ± 0.008	1.000	1.000	0.994
No retrieval	FinBench	3/3	1.000	0.967 ± 0.031	1.000	1.000	0.967
No rewrite	FinBench	3/3	1.000	0.969 ± 0.023	1.000	1.000	0.969
Full PIPE-Cypher	FinBench	3/3	1.000	0.975 ± 0.016	1.000	1.000	0.975
Reverse-only	FinBench	3/3	1.000	0.994 ± 0.011	1.000	0.994	0.994
Validators+repair	FinBench	3/3	1.000	0.987 ± 0.023	1.000	1.000	0.987
No LLM judge	SNB	3/3	1.000	0.982 ± 0.015	1.000	0.996	0.982
No retrieval	SNB	3/3	1.000	0.993 ± 0.004	1.000	0.996	0.993
No rewrite	SNB	3/3	1.000	0.983 ± 0.021	1.000	0.996	0.983
Full PIPE-Cypher	SNB	3/3	1.000	0.987 ± 0.014	1.000	0.996	0.987
Reverse-only	SNB	3/3	1.000	0.989 ± 0.011	1.000	0.996	0.989
Validators+repair	SNB	3/3	1.000	0.987 ± 0.014	1.000	0.996	0.987

Table 11: Target-size and repeated-seed ablation sensitivity. Target coverage normalizes accepted examples by each suite's planned graph/category target, so target-50 and target-100 suites can be compared without treating larger raw counts as quality gains. Unconstrained local generation is excluded from this stability table and reported separately as the attempt-logged stress baseline in Table 9.

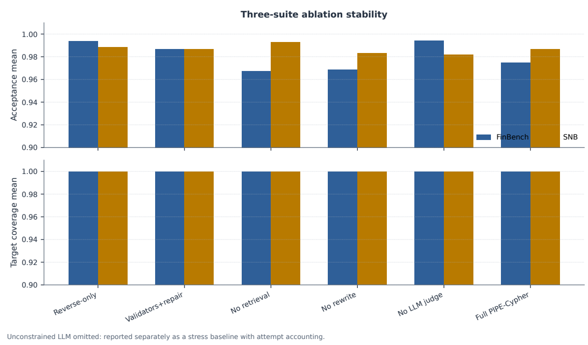

Figure 4: Three-suite ablation stability over the original target-50 suite, corrected target-100 suite, and seed-17 target-50 repeat. Target-normalized coverage stays at 1.000 for all non-unconstrained cells.

B.4 Rejected Candidate Taxonomy

The rejection taxonomy explains where the remaining work occurs. After deterministic Cypher governance, schema-invalid candidates are rare; most rejected candidates are duplicate/diversity blocks from recovery or empty-result candidates caught before export. This is the behavior an enterprise deployment wants: invalid or brittle examples remain in the ledger, not in the benchmark.

Failure bucket	Count	Share of rejected
Diversity/duplicate control	1,366	0.710
Empty result	486	0.252
Judge semantic reject	67	0.035
Execution error	4	0.002
Schema invalid	2	0.001

Table 12: Failure taxonomy over full-run generation candidates before benchmark export. Accepted examples are excluded from the bucket shares.

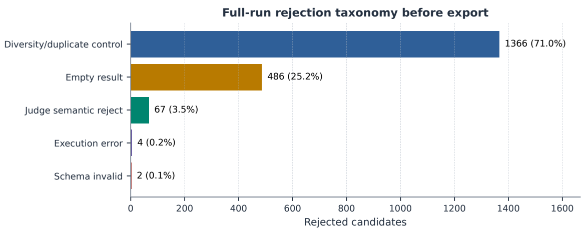

Figure 5: Full-run rejection taxonomy before benchmark export. Most rejected candidates come from duplicate/diversity control during category recovery and from empty execution results; schema-invalid Cypher is rare after deterministic governance.

C Benchmark Artifact and Third-Graph Onboarding

C.1 Export Balance

The exported artifact makes scale and balance visible. Figure 6 shows the planned 2:1 FinBench/SNB mix, exact category balance, and near-even difficulty split. PIPE-Cypher produces a benchmark that is balanced by design rather than sampled opportunistically.

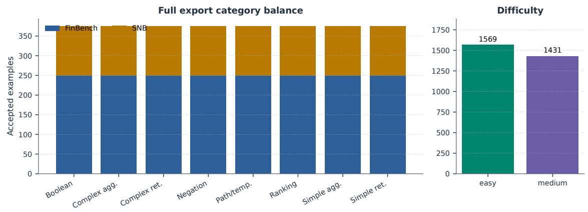

Figure 6: Full 3,000-example export distribution. The benchmark preserves the planned 2:1 FinBench/SNB graph mix while balancing all eight Cypher workload categories and maintaining a near-even easy/medium difficulty split.

C.2 Graph Scale and Schema Variety

Table 13 anchors the graphs we use before introducing the third-graph onboarding audit. FinBench and SNB are the controlled LDBC workloads; ICIJ is the public finance/compliance graph used to test whether the same onboarding machinery works outside the two original schemas.

Graph	Nodes	Relationships	Labels	Rel. types	Study status
FinBench	10,006	57,622	5	9	reported
SNB	34,735	70,842	14	15	reported
ICIJ Offshore Leaks	2,016,523	3,339,267	5	14	onboarding audit

Table 13: Study graph statistics. ICIJ Offshore Leaks is used as a public third-graph onboarding audit for arbitrary finance/compliance schemas beyond the two LDBC study workloads.

C.3 ICIJ Onboarding

Table 14 and Figure 7 report the third-graph result. ICIJ matters because it forced schema-derived sparse-category augmentation rather than relying on preauthored FinBench/SNB templates. The run reaches all eight category targets while preserving the same validation and audit standards.

ICIJ onboarding property	Value
Graph nodes / relationships	2,016,523 / 3,339,267
Labels / relationship types	5 / 14
Generated / accepted	983 / 800
Acceptance rate	0.814
Categories at target	8/8
Study audit	ready
Sparse schema-derived accepts	complex agg. 97, negation 28, ranking 98

Table 14: ICIJ Offshore Leaks third-graph onboarding audit. The public finance/compliance graph tests arbitrary-schema generation beyond the two LDBC study workloads; raw values remain outside the reported artifacts.

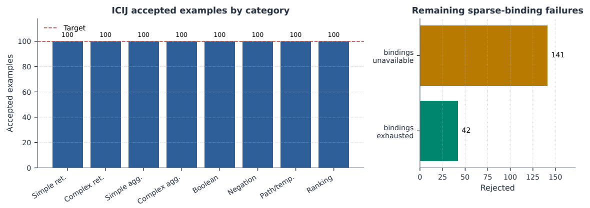

Figure 7: ICIJ Offshore Leaks onboarding audit. Schema-derived sparse-category templates recover balanced target-100 coverage on a public finance/compliance graph beyond the two LDBC workloads.

C.4 Category Crosswalk

Table 15 connects PIPE-Cypher's eight categories to the simpler retrieval/aggregation/evaluation-query taxonomy used in Mind the Query. The crosswalk keeps the comparison legible while making the extra enterprise workload classes explicit: negation, temporal/path reasoning, and ranking/top-k.

PIPE-Cypher category	Closest MTQ category	Added enterprise distinction
Simple retrieval	SR	Direct node/edge lookup with exact filters.
Complex retrieval	CR	Multi-hop or multi-pattern retrieval.
Simple aggregation	SA	Single aggregation such as count/min/max/average.
Complex aggregation	CA	Grouped or multi-stage aggregation over graph neighborhoods.
Boolean existence	EQ	Precise yes/no or existence answer.
Negation/difference	CR	Absence, anti-join, or difference query.
Path/temporal transaction	CR/CA	Temporal or path-oriented transaction neighborhood.
Ranking/top-k	SA/CA	Ordered top-k query with explicit limit.

Table 15: Category crosswalk to Mind the Query. PIPE-Cypher keeps the familiar retrieval/aggregation/evaluation-query structure while adding enterprise workloads such as negation, temporal paths, and ranking.

D Downstream Transfer and Example-Bank Utility

D.1 Transfer Controls

The downstream experiment tests benchmark utility. A useful enterprise benchmark should not merely reward syntactically valid Cypher; it should reveal when a model produces a query that parses, mentions plausible schema elements, and even executes, but answers the wrong operational question. The Qwen3.5-9B result has that shape: parse validity is 0.963 and schema validity is 0.916, while exact execution accuracy is only 0.189 on the full held-out split. The gap shows that PIPE-Cypher measures semantic graph-query competence rather than only query formatting.

The multi-model transfer suite compares local general instruction, code-tuned, Cypher instruction, and Text2Cypher-finetuned models while keeping the schema prompt, evaluation backend, split, and execution metrics fixed. Figure 8 shows a sharp pattern: zero-shot transfer is weak, but accepted graph-specific examples can become a useful private question-answer bank. Demonstration-bank controls close much of the gap for compatible model families such as Qwen, Qwen-Coder, and one Gemma Text2Cypher LoRA, while several public fine-tuned checkpoints remain brittle under enterprise-style prompting.

Table 17 reports checkpoint-level uncertainty for the same controls. The interval is deliberately conservative because the unit is model checkpoint rather than question row; it supports the narrower claim that example-bank gains are model-family dependent, not universal.

Control	Mean acc.	Δ vs zero	95% Δ CI	Models improved
Ordered same-category	0.269	0.233	[0.000, 0.481]	3/11
Scored no-signature	0.200	0.163	[0.000, 0.335]	3/11
Random same-category mean	0.267	0.231	[0.000, 0.464]	3/11

Table 17: Model-level paired bootstrap uncertainty for downstream few-shot controls. The unit of resampling is the local checkpoint, not an individual question, so the interval is a conservative check on whether gains are broad across model families. Zero-shot mean execution accuracy is 0.036.

Table 18 is placed with the transfer result because it defines the correct interpretation: ordered and random same-category demonstrations are useful example-bank conditions, while the scored no-signature condition is the stricter leakage-aware control.

Selection mode	Rows	Demos	Sig. match	High sim.	Mean sim.
Ordered	296	1480	0.866	0.389	0.825
No-signature	296	1480	0.000	0.000	0.585
Random	296	1480	0.854	0.409	0.824

Table 18: Few-shot leakage controls for downstream demonstration-bank evaluation. The held-out split has 0 exact train/test question overlaps and 289 train/test query-signature overlaps; selection-mode rates show how often retrieved demonstrations share the test query signature or exceed the 0.90 normalized-question similarity threshold.

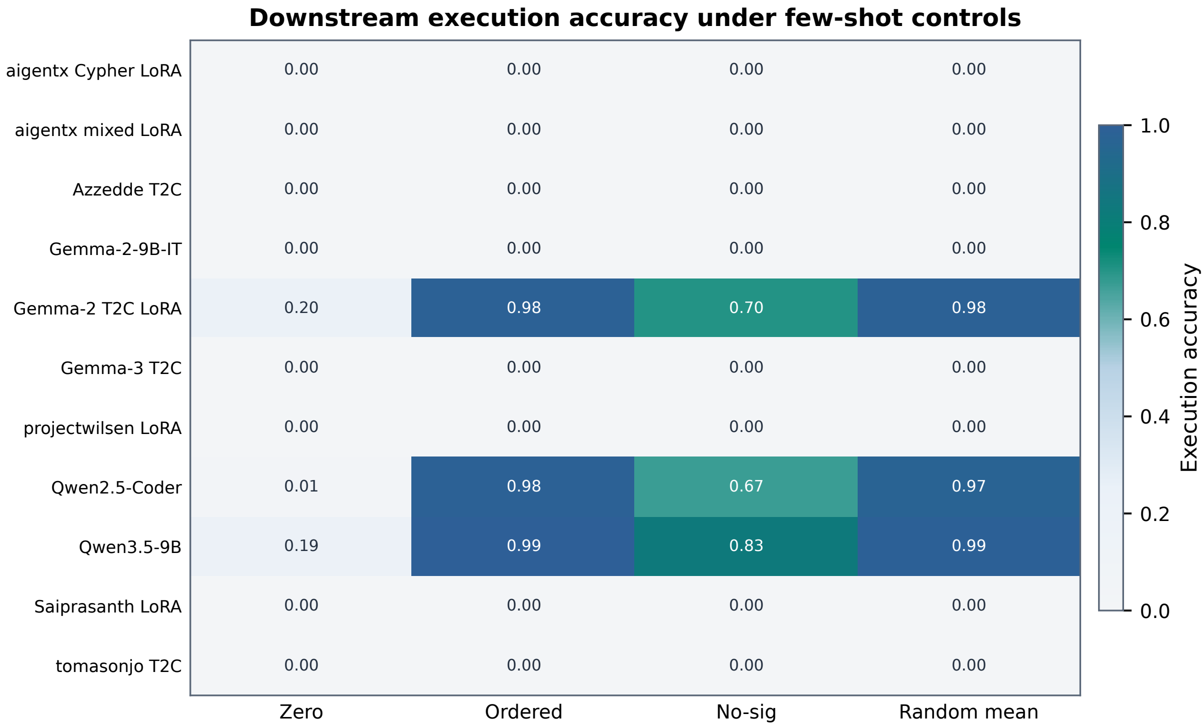

Figure 8: Execution accuracy for 11 completed local downstream models under zero-shot and few-shot control modes. The heatmap highlights both findings: schema-specific examples can sharply help compatible models, and several public fine-tuned checkpoints still fail under enterprise-style Cypher prompting.

Model	Tuning	Zero	Ordered	No-sig	Random μ	Random σ	Best
aigentx/Llama-3.1-8B Cypher LoRA	Cypher LoRA	0.000	0.000	0.000	0.000	0.000	no gain
aigentx/Llama-3.1-8B Cypher mixed LoRA	Cypher mixed LoRA	0.000	0.000	0.000	0.000	0.000	no gain
Azzedde/llama3.1-8b-text2cypher	Text2Cypher fine-tuned	0.000	0.000	0.000	0.000	0.000	no gain
Gemma-2-9B-IT	general instruction	0.000	0.000	0.000	0.000	0.000	no gain
neo4j/Gemma-2-9B Text2Cypher LoRA	Text2Cypher LoRA	0.203	0.983	0.699	0.981	0.009	ordered (0.983)
neo4j/Gemma-3-4B Text2Cypher	Text2Cypher fine-tuned	0.000	0.000	0.000	0.000	0.000	no gain
projectwilsen/Llama-3.1-8B Text2Cypher LoRA	Text2Cypher LoRA	0.000	0.000	0.000	0.000	0.000	no gain
Qwen2.5-Coder-7B-Instruct	code instruction	0.007	0.983	0.669	0.972	0.002	ordered (0.983)
Qwen3.5-9B	general instruction	0.189	0.993	0.828	0.986	0.007	ordered (0.993)
Saiprasanth15/Llama-3.1-8B Text2Cypher LoRA	Text2Cypher LoRA	0.000	0.000	0.000	0.000	0.000	no gain
tomasonjo/text2cypher-demo-16bit	Text2Cypher fine-tuned	0.000	0.000	0.000	0.000	0.000	no gain

Table 16: Few-shot demonstration-bank controls for local downstream Text2Cypher evaluation over completed 296-example local-model runs. Ordered uses the deterministic same-graph, same-category example bank; scored excludes exact query-signature matches and near-duplicate questions; random reports the mean and standard deviation across seeds 13, 17, and 23. "No gain" means no few-shot control exceeded that model's zero-shot execution accuracy.

D.2 Downstream Failure Modes

The failure taxonomy is included next to the transfer results because it explains why the benchmark is useful operationally. Incorrect rows are not all the same: some outputs fail to parse, some mention invalid schema, some fail at execution time, and many execute but answer the wrong question. That separation is what lets a benchmark owner decide whether to improve schema retrieval, prompt constraints, value grounding, or tenant-specific adaptation.

Downstream outcome	Count	Share of incorrect
Answer mismatch	125	0.521
Execution failed	79	0.329
Schema invalid	25	0.104
Parse invalid	11	0.046

Table 19: Downstream Text2Cypher failure taxonomy for local Qwen3.5-9B on the full exported test split. Shares exclude exact-answer matches.

D.3 Supplementary Text Metrics

Table 20 lists optional text-overlap metrics for debugging answer rendering and near-match behavior. These are not correctness metrics; the reported correctness results use execution accuracy and answer-set F1.

Metric	Supported use and primary citation
ROUGE-1/2/L	Reference overlap over serialized answer sets and query strings (Lin, 2004)
BLEU	Sentence-level reference overlap with brevity penalty (Papineni et al., 2002)
METEOR	Exact-token precision/recall with fragmentation penalty (Banerjee and Lavie, 2005)
BERTScore	Optional contextual-embedding similarity (Zhang et al., 2020)
FrugalScore	Optional efficient learned NLG metric (Kamal Eddine et al., 2022)
Cosine	Token-count vector cosine similarity (Manning et al., 2008)
Jaro-Winkler	Character-level near-match similarity (Jaro, 1989; Winkler, 1990)
Exact match	Raw and normalized string exact match (Rajpurkar et al., 2016)

Table 20: Supplementary reference-based text metrics supported by the PIPE-Cypher evaluator. These metrics are useful for debugging answer rendering, paraphrase sensitivity, and near-match behavior, but they do not replace execution accuracy or answer-set F1 for Text2Cypher correctness. BERTScore and FrugalScore are optional integrations because they require additional metric/model packages.

E Diversity and Cypher Strategy Audit

E.1 Diversity Diagnostics

Aggregate acceptance rates hide too much. The diversity and strategy diagnostics show whether the benchmark is balanced, which Cypher operators it exercises, and where residual concentration remains. Figure 9 keeps the useful diversity view: category, graph-category, and difficulty balance are strong by construction, while schema and value coverage remain visible audit quantities.

Metric	Value
PIPE-Diversity index	0.549
Question Distinct-1	0.039
Question Distinct-2	0.085
Question adjusted Distinct-2	0.266
Question self-BLEU-2 (sampled)	0.813
Mean nearest-neighbor question Jaccard	0.775
Unique query-signature ratio	0.041
Top query-signature share	0.083
Template-family entropy	0.826
Operator-combination entropy	0.880
Unique structural substructures	134
Category normalized entropy	1.000
Graph-category normalized entropy	0.980
Difficulty normalized entropy	0.998
Label coverage	0.941
Relationship-type coverage	0.708
Property-name coverage	0.426
Unique grounded-value ratio	0.357
Grounded values exactly quoted	0.823
Aggregation / negation / ordering rates	0.500 / 0.125 / 0.125

Table 21: Diversity diagnostics for the full exported benchmark. PIPE-Diversity is a geometric mean of lexical, query-template, structural, schema, value, and balance components; component rows are shown so the composite score does not hide residual concentration.

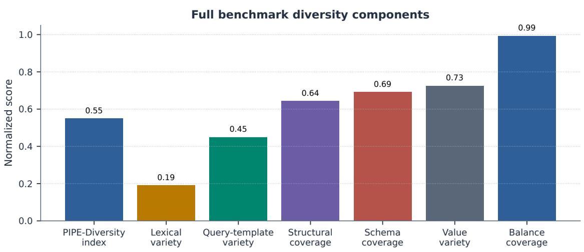

Figure 9: Diversity diagnostics for the full 3,000-example export. Category, graph-category, and difficulty balance are strong by construction; schema coverage and query-signature diversity expose remaining concentration from the seeded-template run.

E.2 Diversity-Governed Selection

Table 22 reports the first diversity-improvement pass: a target-50-per-graph/category selector over the 3,000 accepted full-run examples, compared with a hash-balanced random subset of the same size. The selector uses an MMR-style novelty score over query signatures, template families, structural substructures, schema atoms, entity values, and question tokens after all quality gates have already passed. It improves the PIPE-Diversity index, unique query-signature ratio, unique structural substructures, adjusted Distinct-2, and property coverage while preserving a signature-disjoint 640/80/80 split. The mixed template-family entropy and self-BLEU result is informative: it shows that post-hoc selection can improve coverage, but true diversity gains require oversampling or inducing more source templates during generation when a graph/category cell contains only one or two viable signatures.

Metric	Random balanced	Diversity governed	Δ
PIPE-Diversity index	0.557	0.575	+0.017
Unique query-signature ratio	0.062	0.135	+0.073
Top signature share (lower better)	0.062	0.062	+0.000
Template-family entropy	0.903	0.868	-0.035
Operator-combination entropy	0.901	0.911	+0.010
Unique structural substructures	97.000	134.000	+37.000
Question self-BLEU-2 (lower better)	0.850	0.866	+0.016
Adjusted Distinct-2	0.266	0.284	+0.018
Property coverage	0.407	0.426	+0.019

Table 22: Balanced subset comparison at the same graph/category target. The diversity-governed selector applies MMR-style novelty over Cypher signatures, template families, structural substructures, schema atoms, values, and question tokens after quality gates have already passed; structural/schema gains are reported alongside residual template concentration.

E.3 Cypher Strategy Coverage

Strategy diagnostics complement category balance. Two questions can share a category while exercising different Cypher behavior; conversely, a category-balanced dataset can still miss joins, paths, negation, ranking, or bounded-result patterns. The strategy matrix and downstream strategy outcomes make the benchmark's discriminative structure legible.

Primary strategy	Examples	Share	Rel. patterns	Return arity	Downstream exec. acc.
Aggregation	1125	0.375	1.42	1.00	0.396
Join-heavy	375	0.125	2.33	2.33	0.000
Negation	375	0.125	1.89	2.84	0.000
Order/rank	375	0.125	1.87	3.41	0.000
Path	375	0.125	1.37	3.00	0.000
Single hop	375	0.125	1.00	2.33	0.324

Table 23: Cypher strategy diagnostics over the full 3,000-example export. Strategy tags are derived from generated Cypher structure rather than from category labels; downstream execution accuracy is reported on the full held-out test split when a strategy appears there.

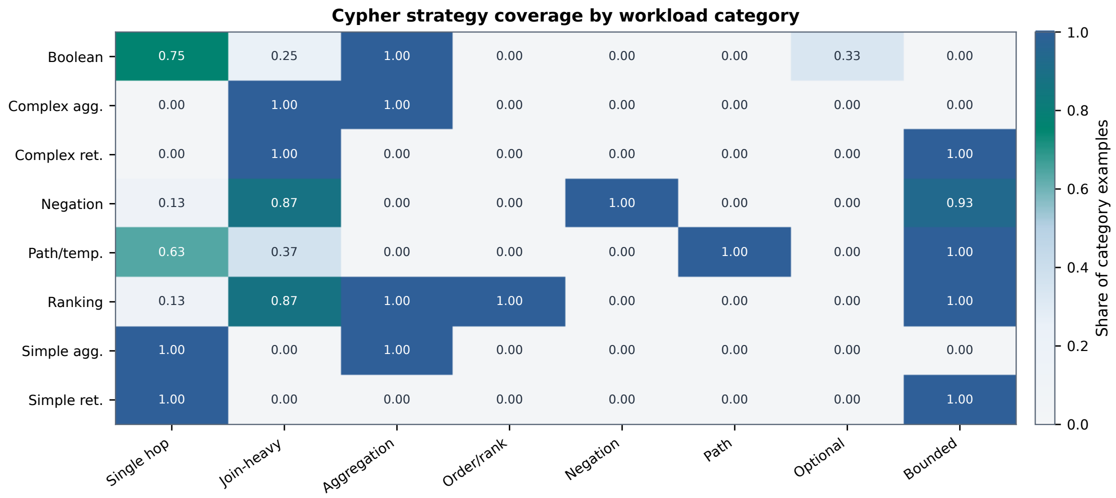

Figure 10: Cypher strategy coverage by workload category. Category balancing does not by itself guarantee operator coverage; the strategy matrix shows where the benchmark exercises single-hop retrieval, joins, aggregation, ordering, negation, path patterns, optional matches, and bounded-result queries.

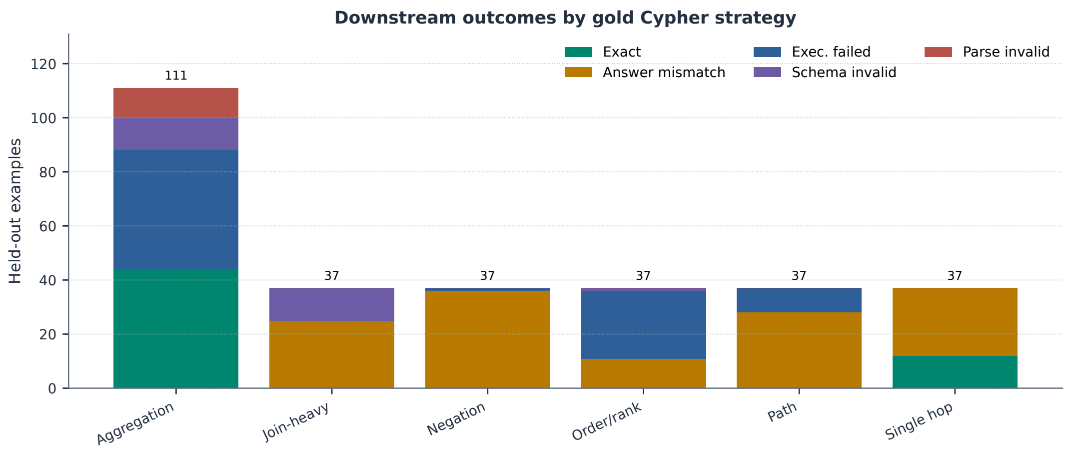

Figure 11: Downstream outcomes by gold Cypher strategy on the full held-out test split. The local Qwen3.5-9B baseline fails differently across strategies: aggregation mixes exact matches with execution failures, while join-heavy, negation, path, and ranking examples are dominated by semantically wrong executable queries or execution failures.

F Governance, Judge Calibration, and Deployment Details

The deployment details below matter because enterprises rarely fail on a single model call. They fail when a benchmark has no safety boundary, no value policy, no audit trail, or no way to explain why an example was accepted. The validator cascade, prompt contracts, automation comparison, and judge audit describe the controls that make PIPE-Cypher a governed local workflow rather than a manual dataset-writing exercise.

F.1 Validation Cascade

Table 24 lists the deterministic and judge gates in execution order. It is the operational view of how an enterprise deployment keeps unsafe, schema-invalid, empty, or semantically weak examples out of exported benchmarks.

Gate or ledger	Count	Denominator
Export accepted	3,000	3,000
Read-only safety	3,000	3,000
Syntax validity	3,000	3,000
Schema/value validity	3,000	3,000
Execution success	3,000	3,000
LLM judge pass	3,000	3,000
Rejected candidates logged	1,925	4,925

Table 24: PIPE-Cypher validation cascade for the full export and its logged candidate ledger. Unlike Mind the Query, human review is calibration-only.

F.2 Rewrite and Governance Audits

Table 25 addresses the rewrite-preservation question directly. In the reported generation records, generated Cypher was already identical to the normalized Cypher, so the accepted benchmark does not rely on a semantics-changing `RETURN DISTINCT` insertion or projection rewrite. The rewrite machinery is still part of the governed pipeline, but its reported-run role is conservative validation and logging rather than silent semantic alteration.

Rewrite audit property	Value
Generation records audited	4,925
Accepted records audited	3,000
Records changed by normalization	0
Accepted records changed	0
RETURN DISTINCT insertions	0
Accepted RETURN DISTINCT insertions	0
Rewrite-skip reasons logged	196
Live comparisons required	0
Answer-set equality in comparisons	0

Table 25: Rewrite prevalence and impact audit over reported generation records. When no generated query differs from its normalized form, no live original/normalized re-execution is required for semantic drift.

Table 26 separates direction, schema/value, syntax/parser, and read-only issues across generation, ablation, and downstream prediction artifacts. The full generation records have no direction failures after governance; downstream predictions still contain direction errors, which is precisely the kind of graph-specific failure that a Cypher benchmark should expose.

Evidence source	Direction	Schema/value	Syntax/parser	Read-only
Full generation records	0	2	0	0
Target-size ablations	260	988	5	0
Downstream predictions	25	9	12	0
Combined	285	999	17	0

Table 26: Governance failure audit. Direction errors, schema/value errors, syntax/parser failures, and read-only violations are counted separately so the appendix shows which Cypher-specific gates do real work.

F.3 Gate-Impact Counterfactual

Table 27 summarizes the first gate that blocks each non-accepted candidate. This is the counterfactual view missing from a saturated yield-only ablation: it shows what kind of bad artifact would enter the benchmark if a deployment weakened duplicate/diversity controls, non-empty execution, judge review, schema validation, direction checking, or read-only safety.

First blocking gate	Candidates	Share
Accepted	3,000	0.609
Duplicate/diversity	1,366	0.277
Empty result	486	0.099
Judge reject	67	0.014
Schema	2	0.000
Execution failure	4	0.001

Table 27: Counterfactual first-blocking-gate audit over generation records. The table shows which failure class would enter the benchmark if that gate were removed or weakened.

F.4 Privacy and Redaction Audit

Table 28 evaluates the redaction policy rather than merely describing it. The audit builds a sensitive-value set from entity bindings, quoted Cypher literals, reverse-grounding literals, and string-valued execution samples, applies the configured redactor, and then exact-matches those raw values against the redacted question, Cypher, entity, and result surfaces. This does not replace a tenant's PII classifier, but it gives reviewers a measurable privacy check for the value-bearing fields PIPE-Cypher itself creates.

Redaction audit property	Value
Examples audited	3,000
Sensitive values checked	10,956
Examples with sensitive values	2,970
Examples with residual raw values	0
Residual raw-value matches	0
Residual rate per checked value	0.000
Unique placeholders	3,754
Reused placeholders	2,342
Max placeholder frequency	337

Table 28: Exact-match redaction audit over value-bearing benchmark surfaces. The audit checks entity bindings, quoted Cypher literals, reverse grounding literals, and string-valued result samples after applying the configured redaction policy.

F.5 Operational Accounting

Table 29 reports local-run accounting from completed generation records. These numbers are included for deployment planning: acceptance rate and graph execution latency explain throughput bottlenecks without converting the analysis into a paid-API cost comparison.

Scope	Records	Accepted	Acceptance	Exec. p50 ms	Exec. p95 ms
Overall	4,925	3,000	0.609	11.995	32.832
FinBench	3,405	2,000	0.587	11.837	32.398
SNB	1,520	1,000	0.658	12.420	38.558

Table 29: Operational accounting from completed generation records. These are local-run latency and acceptance diagnostics, not paid-API cost claims.

F.6 Prompt Profiles and Contracts

Table 30 documents the prompt-profile factors used for ablation planning. The full prompt contracts, including the exact LLM-judge system prompt and user prompt template used for reported judge decisions, follow immediately after the table.

Profile	Added constraint	Intended evidence
Schema-only	Use only visible schema.	Baseline for schema-grounded prompting.
Instructions	Exact values, datatype rules, no nested aggregations.	Targets MTQ-style prompt refinements.
Examples	Placeholderized retrieved NL-Cypher pairs.	Tests whether examples help without extra governance.
Examples + instructions	Few-shot plus explicit rules.	Closest controlled analogue to MTQ Table 5.
Full governed	Production-derived Cypher hints, AST-safe rewrites, judge gate.	PIPE-Cypher production setting.

Table 30: Prompt profiles implemented for Mind-the-Query-style prompt-factorial evaluation. Results are reported only for completed, audited target-50-or-larger suites.

G Prompt Contracts

PIPE-Cypher treats prompts as versioned implementation artifacts. The list summarizes the prompt contracts used for generation, repair, judging, and downstream evaluation; hashes fingerprint the full prompt constants in the codebase.

1. **Template generation.** Stage: Workload proposal. SHA-256: `10b25b`.

   Contract. Schema-only labels, relationships, properties, and categorical values; Realistic enterprise analyst wording; At most two typed slots and JSON-only output

2. **Reverse binding.** Stage: Graph grounding. SHA-256: `48e949`.

   Contract. Read-only MATCH/WHERE/RETURN DISTINCT/LIMIT only; Slot variables named exactly as requested; Forward relationship directions from the schema

3. **Cypher generation.** Stage: Candidate query. SHA-256: `61c557`.

   Contract. Only schema-visible constructs and observed directions; RETURN DISTINCT for set returns and exact equality for quoted values; Context columns, categorical hints, placeholderized retrieval, and no writes

4. **Repair.** Stage: Validation feedback. SHA-256: `56c419`.

   Contract. Preserve question intent while fixing validation or execution issues; Keep query read-only and schema-grounded; Return only corrected Cypher

5. **LLM judge.** Stage: Quality gate. SHA-256: `421c7b`.

   Contract. Inputs include question, Cypher, schema slice, execution rows, and validation summary; Strict JSON scores for ambiguity, semantic alignment, schema use, and difficulty; Categorical values constrain query literals, not observed result-row values; Pass only useful, unambiguous enterprise benchmark examples

6. **Downstream Text2Cypher.** Stage: Model evaluation. SHA-256: `4c07ff`.

   Contract. Read-only Cypher only; Schema-visible constructs and exact direction preservation; RETURN DISTINCT, count/ranking rules, and no explanations

G.1 LLM Judge Prompt Used in Reported Runs

This is the complete LLM-judge prompt contract used for all reported judge decisions. The local judge receives a JSON-only system instruction and the following user prompt template after schema slicing, execution sampling, and deterministic validation. The placeholders are filled with the candidate question, Cypher, schema slice, execution rows, and validation summary for each reviewed example.

```
System prompt:
You are an expert benchmark engineer. Return
strict JSON only. No markdown or extra text.

User prompt template:
You are judging whether an NL-to-Cypher
benchmark example is acceptable for an
enterprise benchmark.

Graph schema:
{schema}

Question:
{question}

Cypher:
{cypher}

Execution sample:
{rows}

Validation summary:
{validation}

Return strict JSON with:
- pass: boolean
- ambiguity_score: number from 0 to 1, lower is
better
- semantic_alignment_score: number from 0 to 1
- schema_use_score: number from 0 to 1
- difficulty: one of easy, medium, hard
- failure_reason: short string, empty if pass is
true

Pass only if the question is unambiguous, the
Cypher answers it, the schema use is valid, and
the result would be useful in an enterprise
benchmark.
- Categorical property values in the schema
constrain literal values written in the Cypher
query.
- Do not reject because the execution sample
returns a value that is absent from the
categorical-value list; result rows are observed
graph outputs.
- If deterministic validation says schema use is
valid, lower schema_use_score only when the
Cypher itself uses nonexistent schema elements,
invalid relationship directions, or invalid
literal values.
```

G.2 Automation and Calibration

Table 31 gives the deployment contrast with Mind the Query, and Table 32 reports the completed human calibration packet. Human review has not disappeared from the research process; it has moved from a production gate to post-hoc calibration evidence.

Dimension	Mind the Query	PIPE-Cypher
Generation review gate	Manual logical review	Deterministic gates + local LLM judge
Human effort	Reported 1,400 person-hours	80-row post-hoc judge calibration audit
Private values	Public benchmark values	Configurable sampling and redacted export
Refresh	Static dataset release	Rerunnable private benchmark factory
Model endpoint	Gemini in their reported pipeline	Local Qwen3.5-9B endpoint

Table 31: Industry deployment contrast with Mind the Query. PIPE-Cypher focuses on private refreshable benchmark generation rather than a one-time public dataset.

Audit packet property	Value
Rows	80
Judge accept / reject	40 / 40
FinBench / SNB rows	48 / 32
Easy / medium rows	26 / 54
Labeled rows	80
Calibration status	complete
Agreement / κ	0.800 / 0.600
Judge precision (95% CI)	1.000 (0.912–1.000)
Judge recall (95% CI)	0.714 (0.585–0.816)
False-accept rate (95% CI)	0.000 (0.000–0.138)
False-reject rate (95% CI)	0.286 (0.184–0.415)

Table 32: Post-hoc judge calibration packet coverage and agreement. Human labels calibrate the automated gate after generation and are not used as a generation gate.

H Representative Accepted Examples

The examples below are selected in stable identifier order from the tracked full-export snapshot, one per graph/category cell when available. They show the natural-language question, accepted Cypher, structural tags, gate status, and a bounded execution-result sample.

1. **FinBench / Boolean Existence / easy.** *Question:* Does account '187743809466009406' have any outgoing transfer?

   ```
   MATCH (src:Account {accountId:
   '187743809466009406'})
   -[:TRANSFER_TO]->
   (:Account)
   RETURN DISTINCT COUNT(DISTINCT src)
   > 0
   AS HasOutgoingTransfer
   ```

   Structure: single_hop, aggregation.

   Relationships: TRANSFER_TO.

   Gates: RO/Syn/Schema/Exec/Judge.

   Result sample: {HasOutgoingTransfer: True}; observed rows: 1

2. **FinBench / Complex Aggregation / medium.** *Question:* What is the total transferred amount from accounts owned by person 'Gwar'?

   ```
   MATCH (p:Person {personName:
   'Gwar'})
   -[:OWN_ACCOUNT]->
   (src:Account)-[t:TRANSFER_TO]->
   (:Account)
   RETURN DISTINCT SUM(t.amount) AS
   TotalTransferredAmount
   ```

   Structure: join_heavy, aggregation.

   Relationships: OWN_ACCOUNT, TRANSFER_TO.

   Gates: RO/Syn/Schema/Exec/Judge.

   Result sample: {TotalTransferredAmount: 14972318.41}; observed rows: 1

3. **FinBench / Complex Retrieval / easy.** *Question:* Which accounts received transfers from accounts owned by person 'Zof'?

   ```
   MATCH (p:Person {personName:
   'Zof'})
   -[:OWN_ACCOUNT]-> (src:Account)
   -[:TRANSFER_TO]-> (dst:Account)
   RETURN DISTINCT dst.accountId AS
   AccountId, dst.accountType AS
   AccountType, dst.isBlocked AS
   IsBlocked
   LIMIT 300
   ```

   Structure: join_heavy, bounded_result.

   Relationships: OWN_ACCOUNT, TRANSFER_TO.

   Gates: RO/Syn/Schema/Exec/Judge.

   Result sample: {AccountId: 4687402787162554854, AccountType: credit card, IsBlocked: False}; observed rows: 3

4. **FinBench / Negation Difference / medium.** *Question:* Which accounts owned by person 'Kant' have not sent any transfers?

   ```
   MATCH (p:Person {personName:
   'Kant'})
   -[:OWN_ACCOUNT]-> (a:Account)
   WHERE NOT (a) -[:TRANSFER_TO]->
   (:Account)
   RETURN DISTINCT a.accountId AS
   AccountId, a.accountType AS
   AccountType,
   a.isBlocked AS IsBlocked
   LIMIT 300
   ```

   Structure: join_heavy, negation, bounded_result.

   Relationships: OWN_ACCOUNT, TRANSFER_TO.

   Gates: RO/Syn/Schema/Exec/Judge.

   Result sample: {AccountId: 4739757132830738341, AccountType: merchant account, IsBlocked: False}; observed rows: 1

5. **FinBench / Path Temporal / medium.** *Question:* Which accounts can receive money within two transfer hops from accounts owned by person 'Sossamon'?

   ```
   MATCH (p:Person {personName:
   'Sossamon'}) -[:OWN_ACCOUNT]->
   (src:Account) -[:TRANSFER_TO*1..2]->
   (dst:Account)
   RETURN DISTINCT dst.accountId AS
   AccountId, dst.accountType AS
   AccountType, dst.isBlocked AS
   IsBlocked
   LIMIT 300
   ```

   Structure: join_heavy, path, bounded_result.

   Relationships: OWN_ACCOUNT, TRANSFER_TO.

   Gates: RO/Syn/Schema/Exec/Judge.

   Result sample: {AccountId: 4687402787162554854, AccountType: credit card, IsBlocked: False}; observed rows: 4

6. **FinBench / Ranking Topk / medium.** *Question:* For accounts owned by person 'Barry', which account sent the highest total transfer amount?

```
MATCH (p:Person {personName:
'Barry'})
-[:OWN_ACCOUNT]->
(src:Account)-[t:TRANSFER_TO]->
(:Account)
WITH src, SUM(t.amount) AS
totalAmount
RETURN DISTINCT src.accountId AS
AccountId, src.accountType AS
AccountType, src.isBlocked AS
IsBlocked,
totalAmount
ORDER BY totalAmount DESC
LIMIT 1
```

Structure: join_heavy, aggregation, order_rank, bounded_result.

Relationships: OWN_ACCOUNT, TRANSFER_TO.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {AccountId: 4732438783436260772, AccountType: internet account, IsBlocked: False}; observed rows: 1

7. **FinBench / Simple Aggregation / easy.** *Question:* How many accounts are owned by person 'Kaewsuktae'?

```
MATCH (p:Person {personName:
'Kaewsuktae'}) -[:OWN_ACCOUNT]->
(a:Account)
RETURN DISTINCT COUNT(DISTINCT a) AS
AccountCount
```

Structure: single_hop, aggregation.

Relationships: OWN_ACCOUNT.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {AccountCount: 1}; observed rows: 1

8. **FinBench / Simple Retrieval / easy.** *Question:* Which accounts are owned by person 'Barry'?

```
MATCH (p:Person {personName:
'Barry'})
-[:OWN_ACCOUNT]-> (a:Account)
RETURN DISTINCT a.accountId AS
AccountId, a.accountType AS
AccountType,
a.isBlocked AS IsBlocked
LIMIT 300
```

Structure: single_hop, bounded_result.

Relationships: OWN_ACCOUNT.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {AccountId: 4732438783436260772, AccountType: internet account, IsBlocked: False}; observed rows: 1

9. **SNB / Boolean Existence / medium.** *Question:* Does person with id 6597069766828 like any post?

```
MATCH (p:Person {id:
6597069766828})
OPTIONAL MATCH (p) -[:LIKES]->
(post:Post)
RETURN DISTINCT COUNT(post) > 0 AS
LikesAnyPost
```

Structure: single_hop, aggregation, optional.

Relationships: LIKES.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {LikesAnyPost: True}; observed rows: 1

10. **SNB / Complex Aggregation / medium.** *Question:* How many distinct posts are in forums joined by person with id 6597069766845?

```
MATCH (forum:Forum) -[:HAS_MEMBER]->
(p:Person {id: 6597069766845})
MATCH (forum) -[:CONTAINER_OF]->
(post:Post)
RETURN DISTINCT COUNT(DISTINCT post)
AS
JoinedForumPostCount
```

Structure: join_heavy, aggregation.

Relationships: CONTAINER_OF, HAS_MEMBER.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {JoinedForumPostCount: 1}; observed rows: 1

11. **SNB / Complex Retrieval / easy.** *Question:* Which people are members of forums containing posts tagged 'Manuel_Noriega'?

```
MATCH (forum:Forum) -[:HAS_MEMBER]->
(p:Person), (forum)
-[:CONTAINER_OF]->
(post:Post) -[:HAS_TAG]-> (tag:Tag
{name: 'Manuel_Noriega'})
RETURN DISTINCT p.id AS PersonId
LIMIT 200
```

Structure: join_heavy, bounded_result.

Relationships: CONTAINER_OF, HAS_MEMBER, HAS_TAG.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {PersonId: 4398046511220}; observed rows: 19

12. **SNB / Negation Difference / easy.** *Question:* Which person records are not linked from any message record through :HAS_CREATOR?

```
MATCH (p:Person)
WHERE NOT EXISTS((:Message)
-[:HAS_CREATOR]-> (p))
RETURN DISTINCT p.id AS PersonId,
p.firstName AS PersonFirstName,
p.lastName AS PersonLastName
```

Structure: single_hop, negation.

Relationships: HAS_CREATOR.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {PersonFirstName: R., PersonId: 8796093022313, PersonLastName: Rao}; observed rows: 25

13. **SNB / Path Temporal / easy.** *Question:* Which people are within two knows hops of person with id 4398046511136?

```
MATCH (src:Person {id:
4398046511136})
-[:KNOWS*1..2]-> (dst:Person)
RETURN DISTINCT dst.id AS PersonId,
dst.firstName AS FirstName,
dst.lastName
AS LastName
LIMIT 200
```

Structure: single_hop, path, bounded_result.

Relationships: KNOWS.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {FirstName: Rafael, LastName: Fernández, PersonId: 4398046511333}; observed rows: 25

14. **SNB / Ranking Topk / medium.** *Question:* Which city records are linked from the most organisation records through :IS_LOCATED_IN?

```
MATCH (s:Organisation)
-[:IS_LOCATED_IN]-> (e:City)
WITH e, COUNT(DISTINCT s) AS
relatedCount
RETURN DISTINCT e.id AS TargetId,
e.name
AS TargetName, relatedCount
ORDER BY relatedCount DESC
LIMIT 10
```

Structure: single_hop, aggregation, order_rank, bounded_result.

Relationships: IS_LOCATED_IN.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {TargetId: 164, TargetName: Kolkata, relatedCount: 130}; observed rows: 10

15. **SNB / Simple Aggregation / easy.** *Question:* How many posts are tagged ’Vietnam’?

```
MATCH (post:Post) -[:HAS_TAG]->
(tag:Tag
{name: 'Vietnam'})
RETURN DISTINCT COUNT(DISTINCT post)
AS
PostCount
```

Structure: single_hop, aggregation.

Relationships: HAS_TAG.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {PostCount: 1}; observed rows: 1

16. **SNB / Simple Retrieval / easy.** *Question:* Which post IDs did person with id 4398046511124 like?

```
MATCH (p:Person {id:
4398046511124})
-[:LIKES]-> (post:Post)
RETURN DISTINCT post.id AS PostId
LIMIT 200
```

Structure: single_hop, bounded_result.

Relationships: LIKES.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {PostId: 343597385744}; observed rows: 5